ENTREGA 2

Testes, Estatísticas e Demonstração dos Agentes

Disciplinas: Arquitetura de Software · Gestão de Projetos · Tecnologias Emergentes

Equipe de 3 (três) integrantes · Entrega exclusivamente em formato .pdf, sem alteração da estrutura do modelo

Capa e Identificação

Campo	Preenchimento
Título do projeto/ecossistema de agentes	Ecossistema de Agentes de Software — Projeto LIMIO (Escola de TI). Agente sob responsabilidade desta equipe: Agente de Validação de Código
Curso/Disciplina/Turma	Bacharelado em Engenharia de Software — UniCesumar — Turma ESOFT-8S · Arquitetura de Software, Gestão de Projetos e Tecnologias Emergentes — 2º semestre de 2026
Integrantes da equipe (3) e papéis	Augusto Ogawa (RA 231759702) — coordenação técnica: arquitetura e implementação do agente Gabriel de Souza Sena (RA 23018106-2) — especificação e qualidade: casos de teste, gabarito e critérios Andrey Eduardo Indras (RA 23050861-2) — gestão do projeto: quadro Kanban, cronograma, riscos e interação com as demais equipes
ODS escolhida	ODS 9 — Indústria, Inovação e Infraestrutura (metas 9.5 e 9.b)
Link/repositório da implementação (se houver)	Código do agente, casos de teste e evidências: pasta agente-validacao-codigo/ do workspace do projeto LIMIO (Java 21 + Maven). Código analisado: github.com/LimioDev/limio-api
Data	10 de novembro de 2026

Nota de escopo. Como registrado na Entrega 1, a dinâmica da AEP foi alterada pelo(a) docente: cada equipe da turma especificou um único agente, e os agentes juntos formam o ecossistema do LIMIO. Esta equipe é responsável pelo **Agente de Validação de Código** (Agente 2 do modelo). As seções dos Agentes 1, 3, 4 e 5 foram mantidas, como exige o modelo, e indicam a equipe responsável por cada um.

1. Resumo Executivo

A equipe implementou o Agente de Validação de Código especificado na Entrega 1: uma ferramenta de linha de comando em Java 21 que recebe o diff de um Pull Request do LIMIO, monta a árvore sintática com JavaParser, aplica 13 regras determinísticas do padrão de camadas do projeto e, só nos trechos suspeitos, consulta um LLM (Claude Sonnet) para encontrar regra de negócio fora do lugar. O agente foi testado em 22 casos com gabarito definido antes da execução: 11 típicos, 7 de borda e 4 PRs reais do repositório limio-api. Cada caso rodou 3 vezes em dois modos, totalizando 132 execuções. Na configuração completa, o agente encontrou os 16 problemas do gabarito sem nenhum falso positivo (precisão e revocação de 100%), com tempo médio de 0,63 s por PR (máximo de 5,62 s, quando a LLM é chamada), contra a meta de 30 s. Esses números são da segunda rodada. A primeira rodada (precisão de 93,3% e revocação de 87,5%) e a varredura do código real revelaram defeitos que foram corrigidos. Por isso, os números da segunda rodada são otimistas e precisam ser confirmados com PRs novos. A principal conclusão: a arquitetura híbrida da Entrega 1

se confirmou, com regras determinísticas como base barata e reprodutível e a LLM reservada ao que nenhuma regra sintática detecta.

2. Ambiente e Metodologia de Teste

Ambiente. Notebook com AMD Ryzen 7 5825U (8 núcleos) e 30 GB de RAM, Linux Mint 22.2 (kernel 6.14), OpenJDK 21, Maven 3.8.7, JavaParser 3.26.2, Jackson 2.17.2, Python 3.12 (script de testes e gráficos) e Git. A camada semântica usa o modelo Claude Sonnet pelo Claude Code 2.1.322 em modo não interativo (claude -p). O código analisado é o limio-api: 305 arquivos .java, 279 deles em src/main.

Dados de entrada. Três fontes: (a) **casos sintéticos**: arquivos reais do limio-api em que um defeito conhecido foi injetado. O script cria um repositório git temporário com o estado "antes" e o "depois" e gera o diff com git diff, como num PR. (b) **PRs reais**: quatro commits do limio-api (6816446, e16d1b9, c25fd14 e 2d6e94c), analisados a partir do próprio git diff. (c) **varredura completa** de src/main do limio-api no modo sob demanda, usada para medir falsos positivos em código que a equipe não escreveu para o teste.

Gabarito. Foi definido antes de rodar o agente. Nos casos sintéticos, cada problema esperado é localizado no arquivo "depois" por um trecho marcador, e não pela saída do agente. Nos PRs reais, o gabarito veio da revisão manual do diff contra o catálogo de regras. Um achado conta como verdadeiro positivo (VP) quando regra, arquivo e linha coincidem. A linha precisa ser exata nas regras determinísticas; para erro de sintaxe (R00) e para os achados da LLM (S01/S02) aceita-se diferença de até 3 linhas. Achado fora do gabarito é falso positivo (FP), e problema do gabarito não apontado é falso negativo (FN).

Métricas e critérios aplicados. Precisão = VP/(VP+FP) e taxa de falsos positivos = FP/(VP+FP), com metas de ≥ 80% e < 20% (critérios 2 e 3 da Entrega 1). Revocação = VP/(VP+FN). Taxa de acerto por caso = casos sem nenhum FP nem FN. Tempo de resposta = tempo de relógio do processo inteiro, incluindo a partida da JVM, com média por caso das 3 execuções e meta de < 30 s (critério 4). Também foram medidos: percentual de achados com arquivo:linha e sugestão (critério 5), consistência entre as 3 execuções, validade do JSON contra o schema do relatório, falhas não tratadas e publicação do evento code-validation-completed. Cada caso rodou em dois modos: **determinístico** (só regras) e **completo** (regras + LLM). Todo o processo é reproduzível com python3 testes/executar.py.

3. Testes, Estatísticas e Demonstração por Agente

Preencha uma seção completa para cada um dos 5 agentes especificados na Entrega 1.

Agente 1 — Agente de Documentação de Software

Confirmação da base tecnológica e do modelo de interação (Entrega 1)

Agente sob responsabilidade de outra equipe da turma, conforme a nota de escopo e a Seção 2 da Entrega 1 (papel esperado: gerar, atualizar e padronizar a documentação técnica do LIMIO; na Entrega 1, consumidor previsto dos achados de nomenclatura e estrutura). A especificação, a base tecnológica e o modelo de interação deste agente pertencem à equipe responsável. Do lado desta equipe, o contrato de integração continua o mesmo da Entrega 1: o Agente de Validação de Código publica o evento code-validation-completed (JSON com o relatório) para consumo assíncrono.

Casos de teste executados (mínimo de 5)

Nº	Cenário testado	Resultado esperado	Resultado obtido
1	Executado pela equipe responsável pelo agente	—	—

2	Executado pela equipe responsável pelo agente	—	—
3	Executado pela equipe responsável pelo agente	—	—
4	Executado pela equipe responsável pelo agente	—	—
5	Executado pela equipe responsável pelo agente	—	—

Métricas e estatísticas coletadas

Métrica	Valor obtido	Meta/critério de aceitação (Entrega 1)
Taxa de acerto/precisão	—	—
Tempo médio de resposta	—	—
Métrica pertinente ao agente	—	—

Gráfico(s) e visualização dos resultados

Os gráficos deste agente constam no relatório da equipe responsável.

Evidência antes/depois

As evidências deste agente constam no relatório da equipe responsável.

Análise crítica do agente

Fora do escopo desta equipe. Ver a Seção 6, "O que não funcionou", sobre a dependência entre equipes.

Agente 2 — Agente de Validação de Código

Confirmação da base tecnológica e do modelo de interação (Entrega 1)

Base tecnológica: mantida. A arquitetura híbrida da Entrega 1 foi implementada como especificada: a camada 1 tem regras determinísticas sobre a árvore sintática (JavaParser) e é a fonte primária das métricas; a camada 2 usa um LLM (Claude) chamado só para trechos suspeitos (controller com decisão if/switch/laço num endpoint, ou classe marcada como God Class). Também foram mantidos o pipeline pipe-and-filter (DiffCollector → ASTParser → Motor de Regras → ReportAggregator) e os padrões Factory Method (RuleAnalyzerFactory, que escolhe as regras pela camada do arquivo), Adapter (RuleSource com DeterministicRuleAdapter e SemanticRuleAdapter), Strategy (13 classes RuleStrategy), Controlador GRASP (ValidationOrchestrator) e o MVC adaptado (Relatorio como Model; JsonView, MarkdownView e TextoView como Views).

Mudanças, com justificativa:

- **LLM pelo Claude Code em vez da API HTTP.** A equipe não tinha chave da API com crédito. O ClaudeCliClient chama claude -p, o mesmo modelo, e fica atrás da interface LlmClient: voltar para a API HTTP troca uma classe, sem mexer no SemanticRuleAdapter. A desvantagem é que o CLI não permite fixar a temperatura, e por isso a consistência foi medida.
- **Checkstyle e PMD não foram usados.** As regras que importam para o LIMIO (camadas actions/controller-usecase-mapper, ADR-0003 entre módulos, Lombok só em entidade) dependem das convenções do projeto e não existem prontas nessas ferramentas. As regras genéricas delas, rodadas no código atual, gerariam ruído sem relação com o catálogo. As 13

regras foram escritas direto sobre a AST. O Adapter RuleSource permite ligar o PMD depois como mais uma fonte de regras.

- **Entregue como CLI (jar), sem o serviço Spring Boot e sem o GitHub App.** Os dois gatilhos da Entrega 1 existem: o modo PR (--diff, que recebe a saída de git diff, como faria um passo de CI) e o modo sob demanda (--caminho). A instalação no GitHub Actions não foi feita nem testada.
- **Modelo de interação: mantido, com transporte simplificado.** O evento code-validation-completed é gravado como arquivo JSON numa pasta de saída (outbox) atrás da interface EventPublisher; trocar por fila ou webhook não muda o orquestrador. Foi publicado em 132 de 132 execuções, mas nenhum agente de outra equipe o consumiu durante os testes.
- **Ajuste de interface:** RuleSource.analisar recebe os achados das fontes anteriores. É assim que a camada semântica sabe se o arquivo é suspeito (ex.: já marcado como God Class) e evita chamar a LLM para todo arquivo.

Casos de teste executados (mínimo de 5)

Foram 22 casos: 14 típicos (C01–C11 e os PRs reais P01–P03) e 8 de borda (B01–B07 e P04). A notação R06@36 significa regra R06 na linha 36. O resultado obtido é o da configuração completa, rodada 2, repetição 1; as três repetições deram o mesmo resultado em todos os casos, exceto a linha apontada pela LLM no C09 (ver consistência).

Nº	Cenário testado	Resultado esperado	Resultado obtido
1 (C01)	Controller injeta Repository (violação de camada) [tipico]	R01@17 (controller acessa repository)	✔️ R01@17
2 (C02)	Classe em actions/usecase sem o sufixo UseCase [tipico]	R05@16 (nomenclatura)	✔️ R05@16
3 (C03)	Consulta JPA dentro de laço (N+1) [tipico]	R06@36 (consulta em laço (N+1))	✔️ R06@36
4 (C04)	Exceção genérica no lugar de NegocioException [tipico]	R07@42 (exceção genérica)	✔️ R07@42
5 (C05)	@RequestBody sem @Valid (Bean Validation não roda) [tipico]	R08@51 (@RequestBody sem @Valid)	✔️ R08@51
6 (C06)	UseCase depende de record e chama Mapper [tipico]	R02@15 (usecase depende de record); R03@13 (usecase chama mapper)	✔️ R03@13; R02@15
7 (C07)	Import direto de outro módulo (fere ADR-0003) [tipico]	R04@10 (import entre módulos)	✔️ R04@10
8 (C08)	Optional.get() sem tratamento (e um get() protegido, que não conta) [tipico]	R10@29 (Optional.get())	✔️ R10@29
9 (C09)	Regra de negócio vazada para o controller (só a camada LLM detecta) [tipico]	S01@55 (regra de negócio no controller (LLM))	✔️ S01@55
10 (C10)	Controller captura exceção de negócio com try/catch [tipico]	R13@53 (try/catch no controller)	✔️ R13@53
11 (C11)	PR com 2 arquivos e 3 violações de tipos diferentes [tipico]	R01@13 (controller acessa repository); R08@30 (@RequestBody sem @Valid); R07@35 (exceção genérica)	✔️ R01@13; R08@30; R07@35
12 (B01)	Diff vazio (PR sem mudanças) [borda]	Nenhum achado	✔️ Nenhum achado (0 arquivo Java analisado)
13 (B02)	Arquivo apenas de teste (src/test) com padrões proibidos em produção	Nenhum achado	✔️ Nenhum achado

	[borda]		
14 (B03)	Somente arquivos não-Java (README e properties) [borda]	Nenhum achado	✔️ Nenhum achado (0 arquivo Java analisado)
15 (B04)	Arquivo Java com erro de sintaxe [borda]	R00@47 (erro de sintaxe)	✔️ R00@47
16 (B05)	Diff malformado (texto que não é diff unificado) [borda]	Nenhum achado	✔️ Nenhum achado + 2 avisos, sem falha
17 (B06)	Arquivo com violação antiga fora das linhas alteradas [borda]	Nenhum achado	✔️ Nenhum achado
18 (B07)	PR que apenas remove um arquivo Java [borda]	Nenhum achado	✔️ Nenhum achado (0 arquivo Java analisado)
19 (P01)	PR real 6816446: endpoints de candidaturas (controller, mapper, records) [tipico]	Nenhum achado	✔️ Nenhum achado
20 (P02)	PR real e16d1b9: use cases de listar e desistir de candidaturas [tipico]	R06@78 (consulta em laço (N+1))	✔️ R06@78
21 (P03)	PR real c25fd14: finders, enum e helper da candidatura [tipico]	Nenhum achado	✔️ Nenhum achado
22 (P04)	PR real 2d6e94c: somente testes (716 linhas) [borda]	Nenhum achado	✔️ Nenhum achado

Métricas e estatísticas coletadas

Valores da rodada 2, a versão final do agente. As colunas trazem a configuração completa e, entre parênteses, só a camada determinística. Os números da rodada 1 estão na análise crítica.

Métrica	**Valor obtido**	**Meta/critério de aceitação (Entrega 1)**
Taxa de acerto/precisão	Precisão 100,0% (16 VP, 0 FP) · (determinístico: 100,0%, 15 VP) Casos 100% corretos: 22/22 (100,0%) · (det.: 21/22)	Critério 2: precisão ≥ 80% — **atendido**
Tempo médio de resposta	0,63 s por PR (desvio padrão 1,07 s; mín. 0,28 s; máx. 5,62 s) · (det.: 0,40 s, desvio 0,06 s) Com chamada à LLM (C09): 5,50 s	Critério 4: < 30 s por PR — **atendido**
Taxa de falsos positivos	0,0% (0 FP em 16 achados) · varredura do limio-api: 0 FP em 7 achados	Critério 3: < 20% — **atendido**
Revocação (recall)	100,0% (16/16) · (det.: 93,8%; o FN é o C09, que só a LLM detecta)	Sem meta na Entrega 1 (métrica complementar)
Achados com arquivo:linha e sugestão	16/16 (100%)	Critério 5: 100% — **atendido**
Casos de borda tratados corretamente	8/8 (100%)	Critério 1: ≥ 5 casos com típicos e borda — **atendido** (22 casos)
Consistência entre 3 execuções	Achados determinísticos: 22/22 casos idênticos · completo: 21/22 (no C09 a LLM apontou a linha 54 numa execução e 55 nas outras)	Mesma entrada, mesmo conjunto determinístico — **atendido**
JSON válido conforme schema / falhas não tratadas	66/66 (completo) e 66/66 (det.) · 0 falhas em 132 execuções, inclusive diff malformado e Java inválido	Saída sempre válida e sem falhas não tratadas — **atendido**

Gráfico(s) e visualização dos resultados

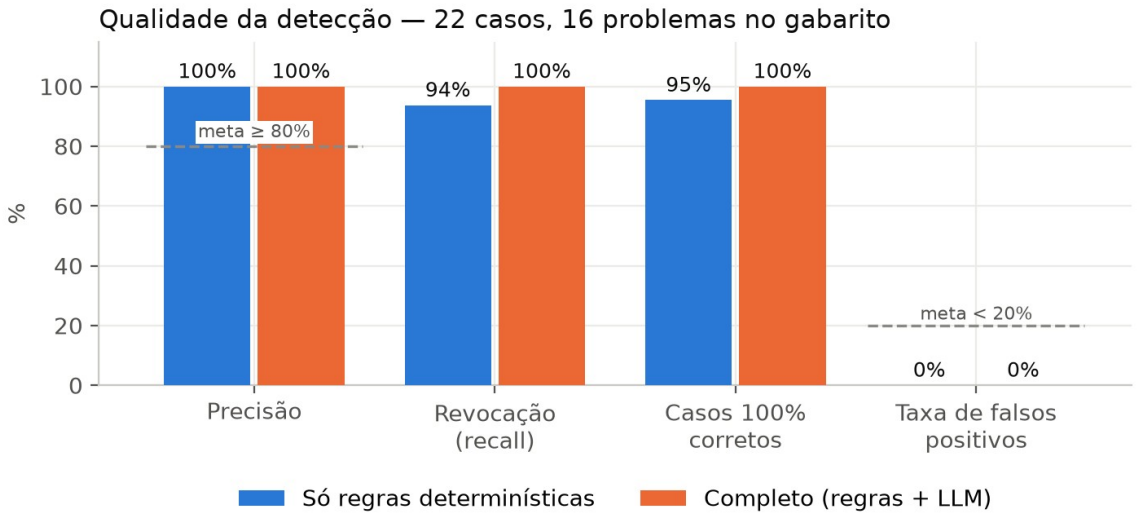


Figura 1 — Precisão, revocação, casos 100% corretos e taxa de falsos positivos nos dois modos (rodada 2). Linhas tracejadas: metas da Entrega 1.

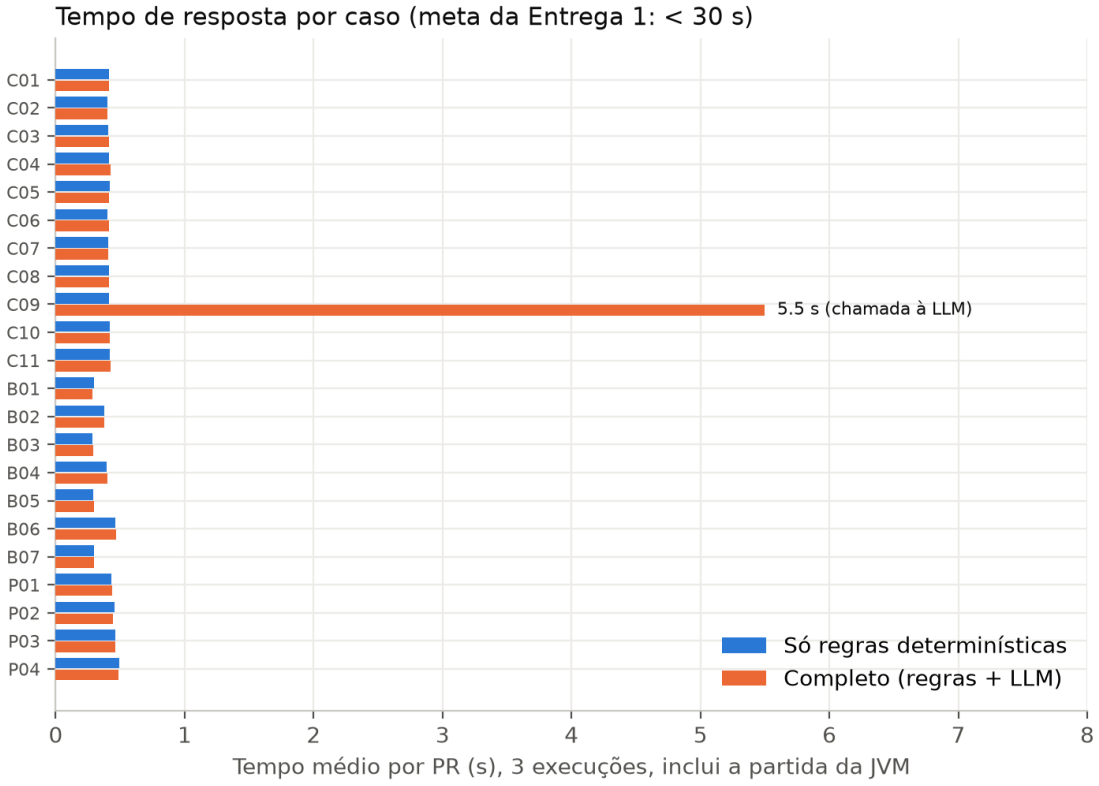


Figura 2 — Tempo médio por caso (3 execuções). Só o C09 aciona a LLM; os demais ficam abaixo de 0,6 s, quase todo esse tempo na partida da JVM (a análise em si levou 0–130 ms).

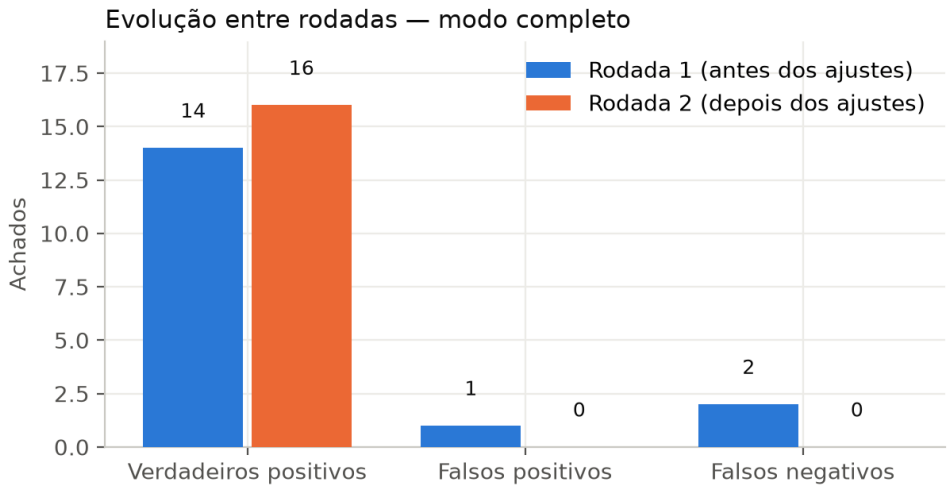


Figura 3 — Verdadeiros positivos, falsos positivos e falsos negativos antes (rodada 1) e depois (rodada 2) dos ajustes, no modo completo.

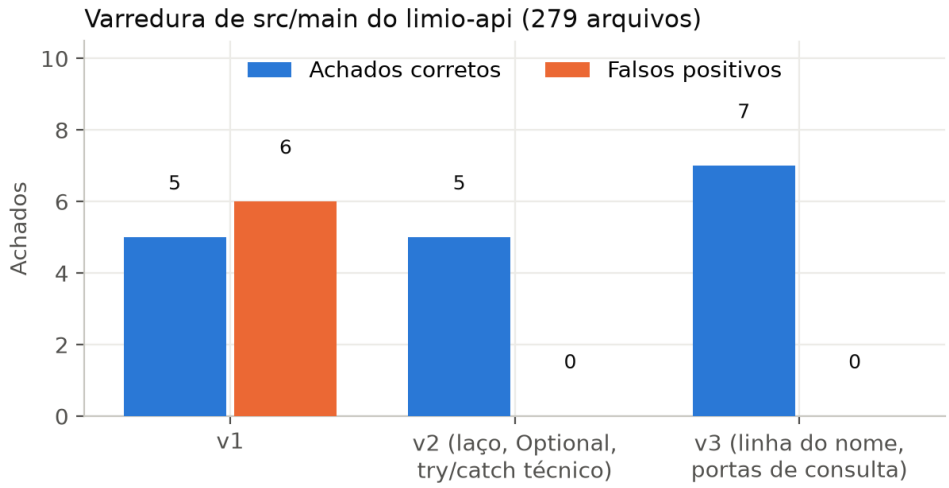


Figura 4 — Varredura de src/main do limio-api em cada versão do agente. Cada achado foi classificado manualmente.

Evidência antes/depois

(a) Atuação do agente sobre um PR (caso C11). Antes: PR com um controller e um use case novos. O agente apontou 3 problemas com arquivo:linha e sugestão, com score 75/100. Depois: as três sugestões foram aplicadas (diff abaixo) e o agente foi executado de novo sobre o código corrigido, com score 100/100 e nenhum achado.

```
$ java -jar agente-validacao.jar --repo pr-c11 --diff pr.diff

.../modulos/anuncio/actions/controller/CandidaturaAdminController.java:13 [ALTA] R01_CONTROLLER_ACESSA_REPOSITORY
    Controller importa o repositório AplicacaoRepository.
    sugestão: Mova o acesso a dados para um UseCase em actions/usecase e injete o UseCase no controller (controller -> usecase -> repository).
.../modulos/anuncio/actions/controller/CandidaturaAdminController.java:30 [ALTA] R08_REQUEST_BODY_SEM_VALID
    Parâmetro 'request' (EncerrarCandidaturaRequest) recebe o corpo da requisição sem @Valid.
    sugestão: Anote o parâmetro com @Valid (jakarta.validation.Valid) para que o Bean Validation do record seja executado antes do UseCase.
.../modulos/anuncio/actions/usecase/EncerrarCandidaturaAdminUseCase.java:35 [MEDIA] R07_EXCECAO_GENERICA
    Captura genérica de Exception.
    sugestão: Crie uma exceção específica em <modulo>/excecao estendendo NegocioException (com CodigoErro) e deixe o GlobalExceptionHandler traduzi-la
       para HTTP.
score 75/100 · 3 achado(s) · 2/2 arquivo(s) Java · 130 ms
```

Figura 5 — Saída real do agente no PR do caso C11 (antes da correção).

```
CandidaturaAdminController.java  (R01 e R08)
-import com.limio.api.modulos.anuncio.aplicacao.AplicacaoRepository;
-    private final AplicacaoRepository aplicacaoRepository;
-    public ResponseEntity<Void> encerrar(@PathVariable UUID id,
-            @RequestBody EncerrarCandidaturaRequest request) {
```

```
+    public ResponseEntity<Void> encerrar(@PathVariable UUID id,
+            @Valid @RequestBody EncerrarCandidaturaRequest request) {

EncerrarCandidaturaAdminUseCase.java  (R07)
-        try {
-            Aplicacao aplicacao = aplicacaoService.buscarParaAtualizar(id).orElseThrow(...);
-            aplicacao.encerrar(Instant.now(), motivo);
-            aplicacaoService.salvar(aplicacao);
-        } catch (Exception e) {
-            log.error("falha ao encerrar candidatura {}", id, e);
-        }
+        Aplicacao aplicacao = aplicacaoService.buscarParaAtualizar(id)
+                .orElseThrow(CandidaturaNaoEncontradaException::new);
+        aplicacao.encerrar(Instant.now(), motivo);
+        aplicacaoService.salvar(aplicacao);
```

```
$ java -jar agente-validacao.jar --repo pr-c11-corrigido --caminho .../anuncio/actions

score 100/100 · 0 achado(s) · 2/2 arquivo(s) Java · 127 ms
```

Figura 6 — Saída real do agente sobre o código corrigido (depois).

(b) Camada LLM (caso C09). A regra "não desistir com menos de 24 h" foi colocada no controller. Nenhuma regra sintática detecta isso; a camada semântica apontou o problema e sugeriu movê-lo para o DesistirCandidaturaUseCase com uma exceção de negócio:

```
$ java -jar agente-validacao.jar --repo pr-c09 --diff pr.diff --semantico

.../modulos/anuncio/actions/controller/MinhasCandidaturasController.java:55 [ALTA] S01_LOGICA_DE_NEGOCIO_NO_CONTROLLER
    Regra de negócio (proibir desistência com menos de 24h para o serviço) calculada no controller, com Duration/Instant.now() e decisão de status 422
      direto.
    sugestão: Remover o parâmetro dataDoServico e o if do controller. Mover a validação para DesistirCandidaturaUseCase, que deve obter a data do serv
      iço a partir da candidatura/anúncio persistido (não confiar em valor enviado pelo cliente) e lançar uma exceção de negócio (ex.: DesistenciaFora
      DoPrazoException) tratada pelo GlobalExceptionHandler, que a mapeia para 422. O controller fica apenas: desistirCandidaturaUseCase.executar(cand
      idaturaId, usuario.id()); return ResponseEntity.ok().build();
score 90/100 · 1 achado(s) · 1/1 arquivo(s) Java · 5096 ms
```

Figura 7 — Saída real do agente com --semantico no caso C09.

(c) Robustez (caso B05). Um texto que não é diff unificado não derruba o agente: ele emite avisos, um relatório válido e o evento.

```
$ java -jar agente-validacao.jar --repo pr-b05 --diff diff-malformado.txt

aviso: 5 linha(s) do diff não reconhecida(s) e ignorada(s)
aviso: diff sem cabeçalho de arquivo válido
score 100/100 · 0 achado(s) · 0/0 arquivo(s) Java · 0 ms
```

Figura 8 — Saída real do agente com diff malformado.

Análise crítica do agente

Os resultados confirmam a especificação da Entrega 1? Sim, nos cinco critérios de aceitação e nos três de qualidade. O ponto mais importante é a divisão de trabalho entre as camadas. As regras determinísticas acertaram 15 dos 16 problemas, com zero falso positivo, sempre com o mesmo resultado e em menos de 0,5 s. A LLM foi chamada em 1 dos 22 casos e acrescentou o único problema que nenhuma regra pega (C09), ao custo de cerca de 5 s. Esse é o equilíbrio de custo, latência e reprodutibilidade que justificou a arquitetura híbrida.

O que a primeira rodada mostrou. Antes dos ajustes, o agente teve precisão de 93,3% e revocação de 87,5% nos casos, e 6 falsos positivos em 11 achados na varredura do limio-api (Figuras 3 e 4). As causas foram regras escritas a partir da literatura, e não do código do projeto, exatamente o risco registrado na Entrega 1:

- **N+1:** uma chamada no cabeçalho do for (for (x : service.buscar(...))) executa uma vez só, mas era tratada como "dentro do laço". **Optional.get():** um get() logo após if (x.isEmpty()) return; é seguro. **try/catch no controller:** converter a IOException de MultipartFile.getBytes() é técnico, não regra de negócio. As três regras foram corrigidas sem tocar no restante do pipeline. É a vantagem prevista do pipe-and-filter com Strategy: cada regra é uma classe isolada.

- **Localização:** classes e métodos eram apontados na linha da primeira anotação (@Service), e não na linha do nome (C02). Corrigido para apontar o nome, como exige o critério 5.
- **Revocação:** no PR real e16d1b9, um N+1 passou porque a dependência é uma porta de consulta de comum/portas (ADR-0003), e não um *Repository/*Service. A regra passou a reconhecer essas portas. Na varredura, isso revelou um segundo caso real em ConsultarPainelEmpregadorUseCase, que existe (uma consulta por anúncio), mas foi detectado por um motivo impreciso: o flatMap de um Optional foi tratado como laço. A análise ainda é intraprocedural e não segue chamadas entre métodos.

Limitações e ameaças à validade. (1) Os ajustes da rodada 2 foram motivados pelos mesmos casos usados para medir, por isso 100% é um limite superior. Para uma medida sem viés, falta um conjunto novo de PRs, separado do usado na calibração. (2) O gabarito foi montado pela própria equipe. (3) A LLM sem temperatura fixa variou a linha apontada; por isso a tolerância de ±3 linhas e a camada determinística como fonte primária. (4) Os limiares de complexidade (10), God Class (20 métodos, 8 dependências, 400 linhas) vieram da literatura e não foram calibrados com a equipe do LIMIO. (5) Não houve teste da integração com CI nem do consumo do evento por outro agente.

Agente 3 — Agente Arquiteto (a confirmar)

Confirmação da base tecnológica e do modelo de interação (Entrega 1)

Agente sob responsabilidade de outra equipe da turma, conforme a nota de escopo e a Seção 2 da Entrega 1 (papel esperado: recomendar estilos arquiteturais e padrões de projeto para novos módulos do LIMIO). A especificação, a base tecnológica e o modelo de interação deste agente pertencem à equipe responsável. Do lado desta equipe, o contrato de integração continua o mesmo da Entrega 1: o Agente de Validação de Código publica o evento code-validation-completed (JSON com o relatório) para consumo assíncrono.

Casos de teste executados (mínimo de 5)

Nº	Cenário testado	Resultado esperado	Resultado obtido
1	Executado pela equipe responsável pelo agente	—	—
2	Executado pela equipe responsável pelo agente	—	—
3	Executado pela equipe responsável pelo agente	—	—
4	Executado pela equipe responsável pelo agente	—	—
5	Executado pela equipe responsável pelo agente	—	—

Métricas e estatísticas coletadas

Métrica	Valor obtido	Meta/critério de aceitação (Entrega 1)
Taxa de acerto/precisão	—	—
Tempo médio de resposta	—	—
Métrica pertinente ao agente	—	—

Gráfico(s) e visualização dos resultados

Os gráficos deste agente constam no relatório da equipe responsável.

Evidência antes/depois

As evidências deste agente constam no relatório da equipe responsável.

Análise crítica do agente

Fora do escopo desta equipe. Ver a Seção 6, "O que não funcionou", sobre a dependência entre equipes.

Agente 4 — Agente de Gestão de Projeto (a confirmar)

Confirmação da base tecnológica e do modelo de interação (Entrega 1)

Agente sob responsabilidade de outra equipe da turma, conforme a nota de escopo e a Seção 2 da Entrega 1 (papel esperado: apoiar backlog, sprints e indicadores ágeis). A especificação, a base tecnológica e o modelo de interação deste agente pertencem à equipe responsável. Do lado desta equipe, o contrato de integração continua o mesmo da Entrega 1: o Agente de Validação de Código publica o evento code-validation-completed (JSON com o relatório) para consumo assíncrono.

Casos de teste executados (mínimo de 5)

Nº	Cenário testado	Resultado esperado	Resultado obtido
1	Executado pela equipe responsável pelo agente	—	—
2	Executado pela equipe responsável pelo agente	—	—
3	Executado pela equipe responsável pelo agente	—	—
4	Executado pela equipe responsável pelo agente	—	—
5	Executado pela equipe responsável pelo agente	—	—

Métricas e estatísticas coletadas

Métrica	Valor obtido	Meta/critério de aceitação (Entrega 1)
Taxa de acerto/precisão	—	—
Tempo médio de resposta	—	—
Métrica pertinente ao agente	—	—

Gráfico(s) e visualização dos resultados

Os gráficos deste agente constam no relatório da equipe responsável.

Evidência antes/depois

As evidências deste agente constam no relatório da equipe responsável.

Análise crítica do agente

Fora do escopo desta equipe. Ver a Seção 6, "O que não funcionou", sobre a dependência entre equipes.

Agente 5 — Agente de Testes Automatizados (a confirmar)

Confirmação da base tecnológica e do modelo de interação (Entrega 1)

Agente sob responsabilidade de outra equipe da turma, conforme a nota de escopo e a Seção 2 da Entrega 1 (papel esperado: gerar e executar casos de teste e reportar cobertura; na Entrega 1, consumidor previsto dos achados para priorizar testes). A especificação, a base tecnológica e o modelo de interação deste agente pertencem à equipe responsável. Do lado desta equipe, o contrato de integração continua o mesmo da Entrega 1: o Agente de Validação de Código publica o evento code-validation-completed (JSON com o relatório) para consumo assíncrono.

Casos de teste executados (mínimo de 5)

Nº	Cenário testado	Resultado esperado	Resultado obtido
1	Executado pela equipe responsável pelo agente	—	—
2	Executado pela equipe responsável pelo agente	—	—
3	Executado pela equipe responsável pelo agente	—	—
4	Executado pela equipe responsável pelo agente	—	—
5	Executado pela equipe responsável pelo agente	—	—

Métricas e estatísticas coletadas

Métrica	Valor obtido	Meta/critério de aceitação (Entrega 1)
Taxa de acerto/precisão	—	—
Tempo médio de resposta	—	—
Métrica pertinente ao agente	—	—

Gráfico(s) e visualização dos resultados

Os gráficos deste agente constam no relatório da equipe responsável.

Evidência antes/depois

As evidências deste agente constam no relatório da equipe responsável.

Análise crítica do agente

Fora do escopo desta equipe. Ver a Seção 6, "O que não funcionou", sobre a dependência entre equipes.

4. Consolidação Estatística Geral do Ecossistema

Agente	Nº de testes	Taxa de sucesso	Tempo médio (s)

Agente de Documentação	Outra equipe	—	—
Agente de Validação de Código	22 casos × 3 repetições × 2 modos = 132 execuções	100% dos casos (completo) · 95,5% (det.)	0,63 (completo) · 0,40 (det.)
Agente 3 — Arquiteto	Outra equipe	—	—
Agente 4 — Gestão de Projeto	Outra equipe	—	—
Agente 5 — Testes Automatizados	Outra equipe	—	—

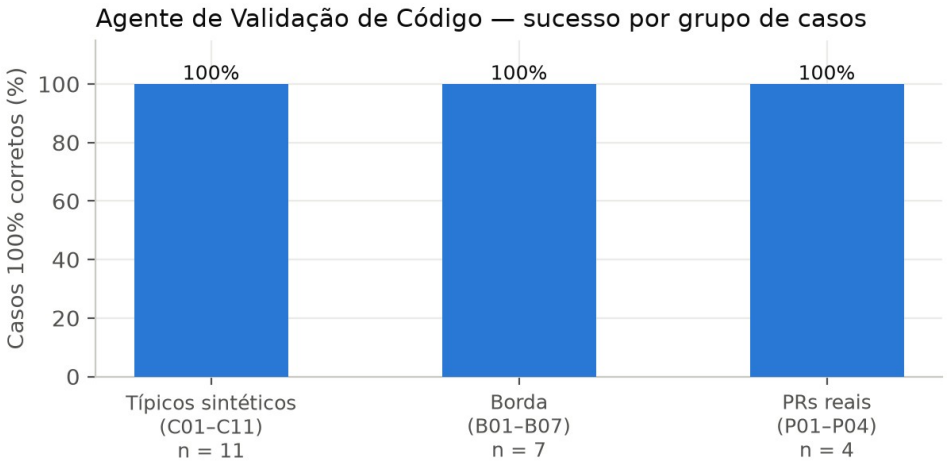


Figura 9 — Agente de Validação de Código: casos 100% corretos por grupo (modo completo, rodada 2). O comparativo com os demais agentes depende dos dados das outras equipes.

5. Demonstração dos Resultados

As Figuras 5 a 8 são saídas reais do agente, capturadas do terminal. A Figura 10 mostra a primeira versão do agente varrendo todo o limio-api: 279 arquivos em 0,75 s, com os 6 falsos positivos que motivaram os ajustes. Todos os artefatos de cada execução ficam em testes/rodada1 e testes/rodada2: estados antes e depois, pr.diff, gabarito.json, relatorio.json, relatorio.md (comentário de PR) e o evento code-validation-completed.json. Para repetir a demonstração:

```
mvn package                                  # gera target/agente-validacao.jar
# modo sob demanda: todo o src/main do limio-api
java -jar target/agente-validacao.jar --repo ../limio-api --caminho src/main/java
# modo PR: diff de um commit real, saída Markdown, com a camada LLM
git -C ../limio-api diff e16d1b9^ e16d1b9 > pr.diff
java -jar target/agente-validacao.jar --repo <checkout> --diff pr.diff \
     --formato markdown --semantico
# bateria completa: 22 casos x 3 repetições x 2 modos + métricas
python3 testes/executar.py
```

```
$ java -jar agente-validacao.jar --repo limio-api --caminho src/main/java   # versão 1

.../comum/excecao/NegocioException.java:14 [BAIXA] R09_LOMBOK_FORA_DE_ENTITY
    Classe 'NegocioException' usa @Getter sem ser entidade JPA.
.../comum/midia/ImagemHelper.java:55 [MEDIA] R11_COMPLEXIDADE_CICLOMATICA
    Método 'processar' tem complexidade ciclomática 12 (limite 10).
.../comum/midia/ImagemHelper.java:232 [MEDIA] R11_COMPLEXIDADE_CICLOMATICA
    Método 'orientacaoNoTiff' tem complexidade ciclomática 11 (limite 10).
.../modulos/anuncio/actions/controller/AnuncioController.java:81 [MEDIA] R13_TRY_CATCH_NO_CONTROLLER
    Controller trata exceção com try/catch.
.../modulos/anuncio/actions/usecase/EncerrarAnunciosEAplicacoesUseCase.java:148 [MEDIA] R10_OPTIONAL_GET_SEM_TRATAMENTO
    Optional.get() chamado sem tratamento da ausência.
.../modulos/anuncio/actions/usecase/EncerrarAnunciosEAplicacoesUseCase.java:158 [ALTA] R06_CONSULTA_EM_LACO_N_MAIS_1
    Chamada aplicacaoService.buscarAbertasDoAnuncio(...) executada a cada iteração (padrão N+1).
.../modulos/anuncio/actions/usecase/EncerrarAnunciosEAplicacoesUseCase.java:186 [ALTA] R06_CONSULTA_EM_LACO_N_MAIS_1
    Chamada reagendamentoService.buscarPendentesDosAnuncios(...) executada a cada iteração (padrão N+1).
.../modulos/auth/actions/controller/AuthController.java:51 [MEDIA] R12_GOD_CLASS
    Classe 'AuthController' concentra responsabilidades: 15 dependências injetadas (limite 8).
.../modulos/auth/actions/helper/IpHelper.java:19 [MEDIA] R11_COMPLEXIDADE_CICLOMATICA
    Método 'ehPublico' tem complexidade ciclomática 13 (limite 10).
.../modulos/auth/tentativalogin/TentativaLoginService.java:60 [MEDIA] R10_OPTIONAL_GET_SEM_TRATAMENTO
    Optional.get() chamado sem tratamento da ausência.
.../modulos/perfil/actions/controller/FotoPerfilController.java:53 [MEDIA] R13_TRY_CATCH_NO_CONTROLLER
    Controller trata exceção com try/catch.
score 38/100 · 11 achado(s) · 279/305 arquivo(s) Java · 808 ms
```

Figura 10 — Varredura do limio-api pela versão 1 do agente (as sugestões foram omitidas para caber).

6. Retrospectiva de Gestão do Projeto

Aderência à metodologia planejada

O Kanban da Entrega 1 (A Fazer / Em Andamento / Em Revisão / Concluído) foi mantido. Mudou a natureza dos itens: em vez de seções de documento, o quadro passou a ter os estágios do pipeline (coleta do diff, AST, regras, agregação e views, camada LLM), os casos de teste e o documento. A adaptação mais importante foi um ciclo de medição em duas rodadas: medir, classificar cada erro, corrigir a regra responsável e medir de novo. A coluna "Em Revisão" serviu para revisar à mão os achados do agente no código real, o que não estava no plano original e se mostrou a atividade mais valiosa da entrega.

O que funcionou bem

- **Definir o gabarito antes de executar**, com marcadores no código em vez de números de linha digitados à mão, o que eliminou a tentação de ajustar o esperado ao obtido.
- **Usar o código e os commits reais do LIMIO** como base dos casos. Os erros mais relevantes, como os 6 falsos positivos e o N+1 nas portas de consulta, só apareceram em código real.
- **Automatizar a medição de ponta a ponta** (python3 testes/executar.py). Repetir 132 execuções levou minutos, e a rodada 2 foi feita nas mesmas condições da rodada 1.
- **A arquitetura da Entrega 1 facilitou a manutenção.** Os ajustes tocaram 7 das 13 classes de regra e nenhuma outra parte do pipeline.

O que não funcionou / dificuldades enfrentadas

- **Dependência entre equipes.** A composição do ecossistema não se fechou a tempo, então não foi possível testar o consumo do evento nem reunir os números dos demais agentes nesta entrega. O risco já estava registrado na Entrega 1, mas a mitigação (contrato JSON desacoplado) resolveu só o lado técnico, não o de cronograma.
- **Regras escritas antes de olhar o código.** A primeira versão gerou mais falsos positivos que acertos no código real (6 contra 5), o que levou a retrabalho.
- **Premissas de infraestrutura não verificadas.** A Entrega 1 assumiu acesso à API do Claude e o uso de Checkstyle/PMD. Sem chave de API, foi preciso adaptar o cliente da LLM, e a integração com o GitHub Actions ficou de fora.

Lições aprendidas

- Calibrar com o código real desde a primeira versão e **separar o conjunto de calibração do conjunto de teste**, para que a métrica final não seja otimista.

- Combinar cedo, com as outras equipes, o contrato e o cronograma de integração, com um teste de ponta a ponta como marco do quadro, e não só como suposição documentada.
- Verificar no início do projeto o acesso a cada dependência externa (chave de API, permissões no repositório e na CI).
- Medir também o custo por execução (tokens e chamadas) da camada LLM, além do tempo.

7. Conclusão

A entrega cumpriu o objetivo de aprendizagem da AEP de levar uma especificação arquitetural até um experimento controlado. Os padrões escolhidos na Entrega 1 (pipe-and-filter, Strategy, Adapter, Factory Method, Controlador) foram testados na prática: permitiram corrigir regras isoladas entre as rodadas e trocar o provedor da LLM sem mexer no núcleo. Do ponto de vista de tecnologias emergentes, o resultado mostra onde a LLM agrega valor: no problema semântico que nenhuma regra pega, não na verificação que uma regra faz mais rápido, mais barato e sempre do mesmo jeito. Do ponto de vista de gestão, o ciclo de medição e a revisão manual do código real foram decisivos, e a dependência entre equipes foi o principal risco não resolvido.

Quanto à ODS 9, o agente atua na meta 9.5 (capacidade tecnológica) e na 9.b (desenvolvimento tecnológico local) como mecanismo de manutenção da infraestrutura de software do LIMIO. Ele revisou os 279 arquivos de produção em menos de um segundo e apontou dívidas técnicas reais: duas consultas N+1 em telas de listagem, três métodos acima do limite de complexidade, um controller com 15 dependências e Lombok fora de entidade. Cada uma veio com uma sugestão de refatoração, o que transforma em rotina uma revisão que hoje depende da disponibilidade de três desenvolvedores.

8. Referências Utilizadas

ANDERSON, D. J. Kanban: Successful Evolutionary Change for Your Technology Business. Sequim: Blue Hole Press, 2010.

ANTHROPIC. Claude Code: documentação (modo não interativo, claude -p). Disponível em: https://docs.claude.com/en/docs/claude-code. Acesso em: 10 nov. 2026.

BASS, L.; CLEMENTS, P.; KAZMAN, R. Software Architecture in Practice. 4. ed. Boston: Addison-Wesley, 2021.

BLOCH, J. Effective Java. 3. ed. Boston: Addison-Wesley, 2018.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley, 1994.

JAVAPARSER. JavaParser: documentação e API (versão 3.26). Disponível em: https://javaparser.org. Acesso em: 10 nov. 2026.

LARMAN, C. Utilizando UML e Padrões: uma introdução à análise e ao projeto orientados a objetos. 3. ed. Porto Alegre: Bookman, 2007.

LIMIO. Repositório limio-api: AGENTS.md e ADRs 0002 e 0003. Disponível em: https://github.com/LimioDev/limio-api. Acesso em: 10 nov. 2026.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. Introduction to Information Retrieval. Cambridge: Cambridge University Press, 2008. (cap. 8: precisão e revocação)

MARTIN, R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River: Prentice Hall, 2008.

McCABE, T. J. A Complexity Measure. IEEE Transactions on Software Engineering, v. SE-2, n. 4, p. 308–320, 1976.

ONU. Transformando Nosso Mundo: a Agenda 2030 para o Desenvolvimento Sustentável. Nova York: ONU, 2015.